CanvasLib para Python

Manual de utilização no VS Code · macOS · Edição educativa

Introdução

Esta documentação descreve a versão educativa da CanvasLib para Python, destinada a programas executados no VS Code no Mac. Foi adaptada a partir da organização e das operações da CanvasLib para Kotlin. A implementação utiliza tkinter e não é uma versão oficial da biblioteca Kotlin.

1. Preparar o VS Code

Instala Python 3 e a extensão Python (Microsoft) no VS Code. Cria uma pasta para o projeto e coloca nela os ficheiros canvaslib.py e programa.py. Abre a pasta no VS Code. No terminal, verifica a instalação e o suporte gráfico:

```
python3 --version
python3 -m tkinter
```

2. Primeiro programa

O objeto Canvas representa uma área de desenho. A função run() mantém a janela aberta e trata dos eventos. Em Python, não são necessários onStart nem onFinish.

```
from canvaslib import *

arena = Canvas(400, 300, CYAN)
arena.drawCircle(200, 150, 70, RED, 5)
arena.run()
```

3. Cores e coordenadas

As cores predefinidas são BLACK, WHITE, RED, GREEN, BLUE, YELLOW, CYAN e MAGENTA. Também podes usar valores RGB hexadecimais, como 0xEE82EE. O ponto (0, 0) fica no canto superior esquerdo; x cresce para a direita e y cresce para baixo. A largura e a altura estão disponíveis em arena.width e arena.height.

4. Criar e limpar o Canvas

Canvas(width, height, background=WHITE) cria uma janela. erase() apaga todas as figuras desenhadas; não altera a cor de fundo.

```
arena = Canvas(500, 300, WHITE)
arena.drawCircle(100, 100, 30, BLUE)
arena.erase()
arena.drawCircle(200, 100, 30, GREEN)
arena.run()
```

5. Desenhar linhas

drawLine(xFrom, yFrom, xTo, yTo, color=BLACK, thickness=3) desenha um segmento entre dois pontos.

```
arena = Canvas(400, 250, WHITE)
arena.drawLine(20, 20, 380, 200, RED, 5)
arena.drawLine(20, 200, 380, 20, BLUE)
arena.run()
```

6. Desenhar retângulos

drawRect(x, y, width, height, color=BLACK, thickness=0). Com thickness=0, desenha um retângulo preenchido; com valor positivo, desenha apenas o contorno.

```
arena = Canvas(400, 250, YELLOW)
arena.drawRect(30, 40, 120, 100, BLUE)
arena.drawRect(210, 40, 120, 100, RED, 6)
arena.run()
```

7. Desenhar círculos

drawCircle(xCenter, yCenter, radius, color=BLACK, thickness=0). O centro é indicado pelas coordenadas xCenter e yCenter.

```
arena = Canvas(400, 250, WHITE)
arena.drawCircle(100, 120, 60, GREEN)
arena.drawCircle(280, 120, 60, BLACK, 5)
arena.run()
```

8. Desenhar arcos

drawArc(xCenter, yCenter, radius, startAng, endAng=360, color=BLACK, thickness=0). Os ângulos são medidos em graus; 0 aponta para a direita e os valores positivos seguem o sentido contrário ao dos ponteiros do relógio. Com thickness=0 desenha um setor preenchido.

```
arena = Canvas(400, 250, WHITE)
arena.drawArc(100, 120, 70, 0, 90, GREEN)
arena.drawArc(280, 120, 70, 45, 270, BLUE, 5)
arena.run()
```

9. Desenhar texto

drawText(x, y, txt, color=BLACK, fontSize=None) escreve texto. O ponto (x, y) é a referência inferior esquerda do texto. Se fontSize for omitido, reutiliza o tamanho anterior (ou 32 inicialmente).

```
arena = Canvas(450, 250, WHITE)
arena.drawText(20, 80, "Ola, Python!", BLUE, 28)
arena.drawText(20, 150, "CanvasLib", RED, 36)
arena.run()
```

10. Desenhar imagens

drawImage(filename, x, y, width=0, height=0) apresenta uma imagem. Requer Pillow: python3 -m pip install pillow. A imagem é procurada na pasta de execução; sem extensão assume .png. Também é possível recortar: nome|x,y,largura,altura.

```
from canvaslib import *
arena = Canvas(400, 300, WHITE)
arena.drawImage("bola.png", 30, 30, 100, 100)
arena.drawImage("bola|10,10,50,50", 180, 30, 100, 100)
arena.run()
```

11. Executar código mais tarde

onTime(delay, handler) chama uma função uma vez, após delay milissegundos. Não coloques parênteses após o nome da função quando a passas como argumento.

```
arena = Canvas(450, 200, YELLOW)

def mostrar_mensagem():
    arena.drawText(20, 100, "Passaram 2 segundos!", BLUE, 24)

arena.onTime(2000, mostrar_mensagem)
arena.run()
```

12. Animações periódicas

onTimeProgress(period, handler) chama repetidamente uma função, passando-lhe o tempo decorrido em milissegundos. O objeto devolvido permite parar a repetição com stop().

Exemplo: bola em movimento suave

Neste exemplo, a posição é calculada pelo tempo real decorrido, em vez de somar sempre a mesma distância. A animação é atualizada aproximadamente a cada 16 ms. O valor real pode variar com a carga do computador.

```
from canvaslib import *

arena = Canvas(600, 300, WHITE)
raio = 20
velocidade = 150  # pixeis por segundo

def mover(tempo):
    segundos = tempo / 1000
    percurso = arena.width - 2 * raio
    x = raio + (velocidade * segundos) % percurso
    arena.erase()
    arena.drawCircle(x, 150, raio, RED)

temporizador = arena.onTimeProgress(16, mover)
arena.run()
```

13. Rato: cliques

onMouseDown(handler) executa a função quando se prime o botão esquerdo do rato. O evento contém x, y e down.

```
arena = Canvas(500, 300, WHITE)

def clicar(evento):
    arena.drawCircle(evento.x, evento.y, 15, GREEN)

arena.onMouseDown(clicar)
arena.run()
```

14. Rato: movimento

onMouseMove(handler) executa uma função sempre que o rato se move sobre o Canvas. A propriedade arena.mouse contém o último estado conhecido do rato.

```
arena = Canvas(500, 300, WHITE)

def desenhar(evento):
    cor = RED if evento.down else BLUE
    arena.drawCircle(evento.x, evento.y, 3, cor)

arena.onMouseMove(desenhar)
arena.run()
```

15. Teclado

onKeyPressed(handler) recebe um evento com char (carácter), code (código) e text (nome da tecla). Se necessário, clica primeiro na janela para lhe dar foco.

```
arena = Canvas(500, 200, WHITE)

def tecla(evento):
    arena.erase()
    arena.drawText(20, 100, evento.text, BLUE, 28)

arena.onKeyPressed(tecla)
arena.run()
```

16. Fechar o Canvas

close() fecha a janela e termina o ciclo de eventos. Este exemplo fecha a janela quando o utilizador clica nela.

```
arena = Canvas(400, 200, YELLOW)

def fechar(evento):
    arena.close()

arena.onMouseDown(fechar)
arena.run()
```

17. Sons e limitações

playSound(nome) está implementada apenas para Windows e ficheiros WAV nesta versão. No macOS, a função lança NotImplementedError. loadSounds(*names) existe apenas como compatibilidade e não faz pré-carregamento. A biblioteca foi pensada para usar uma janela Canvas de cada vez. As animações com erase() redesenham toda a área e podem ter pequenas irregularidades; não existe uma API de sprites ou double buffering configurável.

18. Exercícios propostos

1. Desenha uma bandeira com três retângulos.
2. Desenha uma cara com dois olhos e uma boca.
3. Desenha círculos onde o utilizador clicar.
4. Faz uma bola deslocar-se da esquerda para a direita.
5. Faz a bola inverter a direção quando toca na parede.
6. Muda a cor da bola ao premir a tecla espaço.

19. Resolução de problemas

Se surgir ModuleNotFoundError: No module named 'canvaslib', confirma que canvaslib.py está na mesma pasta que o programa. No macOS, executa com python3 programa.py. Se não abrir uma janela, verifica python3 -m tkinter. Para imagens, instala Pillow no mesmo interpretador Python usado pelo VS Code.